TYPESCRIPT CODING CHALLENGE

Rate limiter: design and solution

`rateLimit(consumer)`: at most **10 requests in any 1-second span** per consumer, and the exact milliseconds to wait when denied.

Loic Wong · October 2026 · 14 tasks · 226 tests

THE PROBLEM

Requirements

Core

- `rateLimit(consumer)` decides whether an identified consumer may proceed
- At most **10 requests in any 1-second span**, per consumer
- When denied: **how many ms to wait** before retrying
- A test suite that demonstrates it works

(a) Limits per resource and per consumer Built

`ConfigurablePolicyResolver` : most specific rule wins, validated up front

(b) Highly available web app Designed

Redis design documented in the README; the store and strategy ports are ready for it

Also built

IETF `RateLimit-*` headers + `Retry-After` ; client-side `acquire` and `withRateLimit`

USAGE

Public API at a glance

```ts
import { rateLimit } from './src';

const result = await rateLimit({ consumerId: 'user-42' });

if (result.allowed) {
  // Proceed. result.remaining = quota left.
} else {
  console.log(`Try again in ${String(result.retryAfterMs)} ms`);
}
```

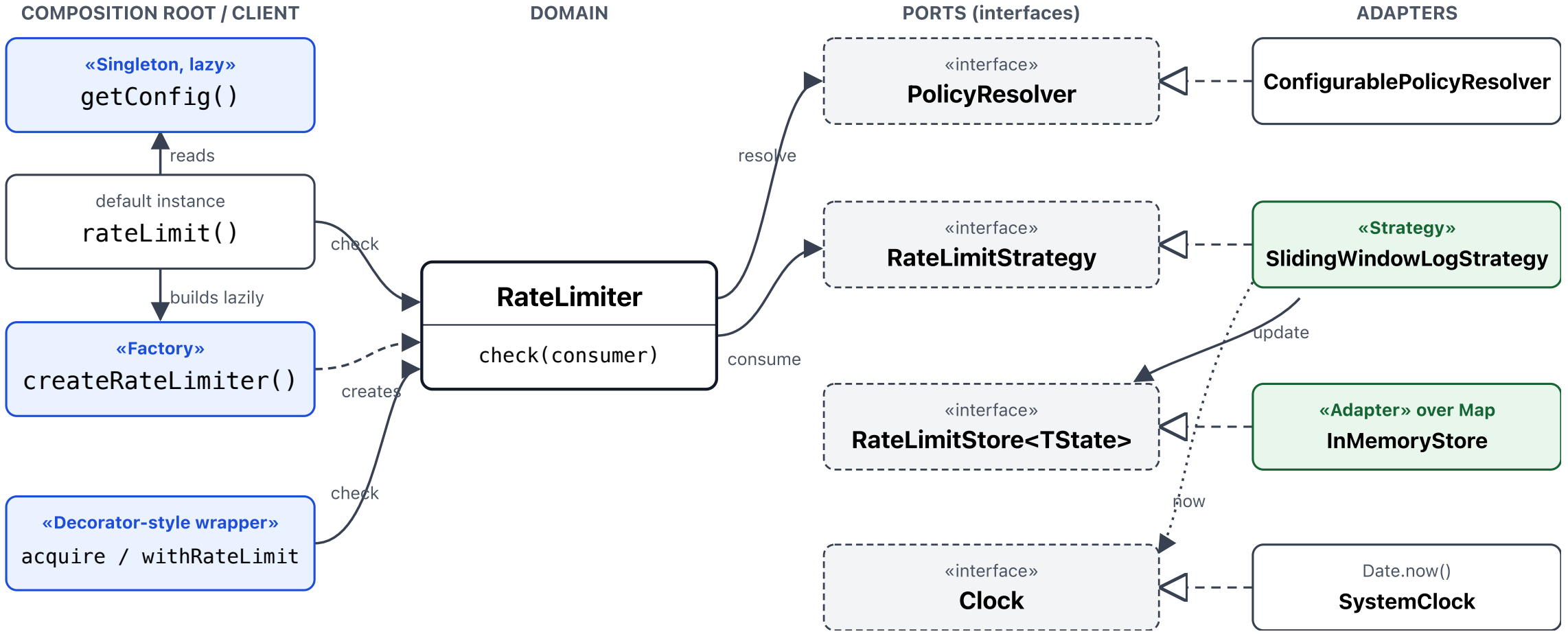
```ts
type RateLimitResult = { limit; resetMs } & (
  | { allowed: true;  remaining; retryAfterMs: 0 }
  | { allowed: false; remaining: 0; retryAfterMs }
);  // discriminated on `allowed`, all readonly
```

Main exports of `src/index.ts`

Function	Purpose
`rateLimit(consumer)`	Default limiter, lazily built from env (10 / 1000 ms)
`createRateLimiter(opts)`	Own limiter: `defaultPolicy` *or* `policies` (+ `clock`, `store`)
`limiter.check(consumer)`	Check against that limiter
`toRateLimitHeaders(r)`	HTTP response headers for a result
`acquire(...)`	Wait for a slot instead of being denied
`withRateLimit(...)`	Wrap a function so each call waits for a slot
`getConfig()`	Validated env config (lazy singleton)

ARCHITECTURE

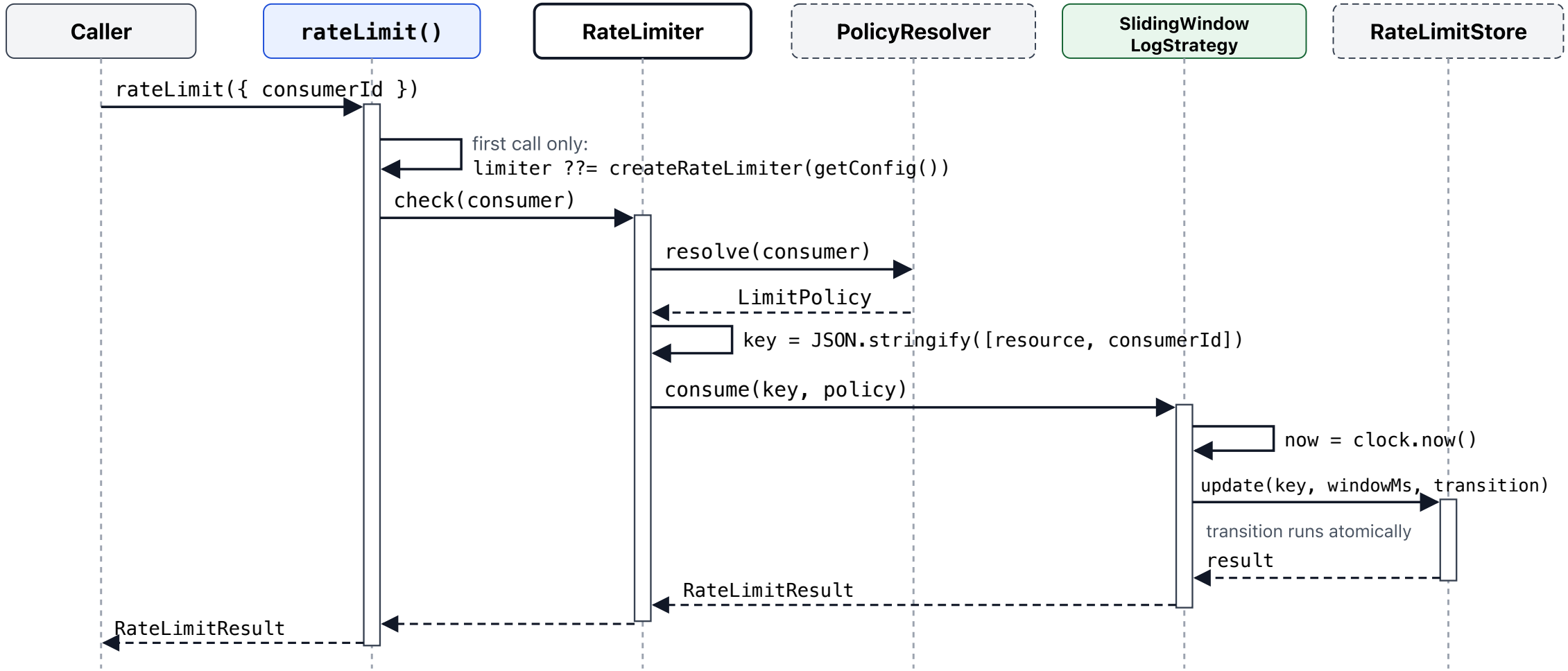
Ports and adapters, one pattern per seam



Solid arrow: uses · dashed triangle: implements · a new variant is a new class; `RateLimiter` never changes (OCP, DIP)

ARCHITECTURE

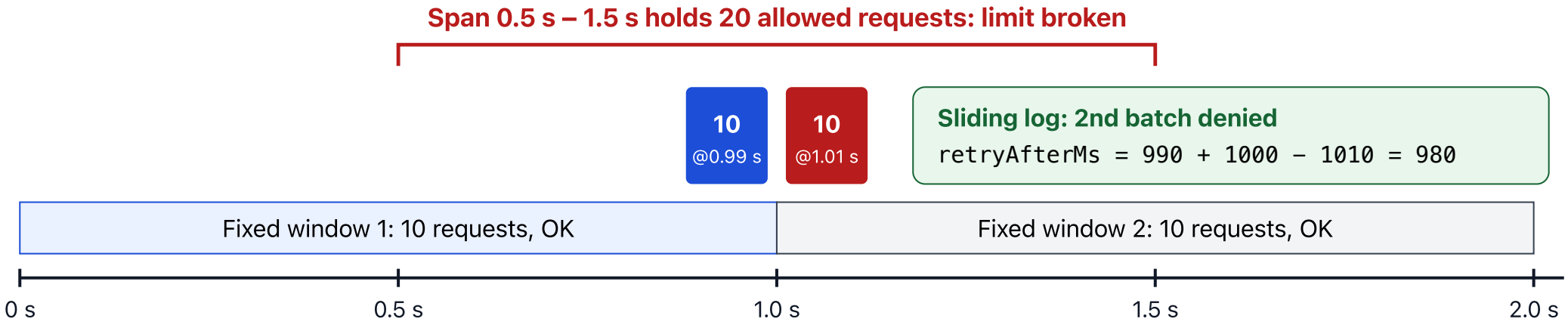
Request flow of one check



ALGORITHM

Why a sliding window log

Algorithm	Holds "≤ 10 in *any* second"?	Memory / consumer	Wait time
Fixed window (counter per whole second)	**No: 20 in 20 ms at a boundary**	O(1)	Coarse (to boundary)
Token bucket (10 tokens, 10/s refill)	**No: up to ~19 in a rolling second**	O(1)	Exact next token
Sliding window log (chosen)	**Yes, exactly, by construction**	O(limit): ≤ 10 numbers	**Exact**: `oldest + 1000 − now`



ALGORITHM

Sliding window log, step by step

```
consume(key, { limit, windowMs }) {
  const now = this.clock.now();
  return this.store.update(key, windowMs, (previous = []) => {
    const live = previous.filter((t) => t > now - windowMs); // 1
    if (live.length < limit) {                                // 2
      const state = [...live, now];
      return { state, result: { allowed: true,
        remaining: limit - state.length, ... } };
    }
    const waitMs = live[0] + windowMs - now;                  // 3
    return { state: live,     // denied: not recorded
      result: { allowed: false, retryAfterMs: waitMs, ... } };
  });
}
```

Simplified from `src/strategies/SlidingWindowLogStrategy.ts`

- **1. Drop** timestamps `≤ now − windowMs`
- **2. Allow + record** `now` if under the limit
- **3. Deny** with the *exact* wait until the oldest leaves
- **Denied requests are not recorded**: hammering never pushes the wait out
- The log never holds more than `limit` timestamps (asserted)
- Atomic: `InMemoryStore` reads, transitions and writes with no `await` in between

Test, 10 per 1000 ms: 5 at t=0 and 5 at t=500 → t=999 denied, `retryAfterMs: 1` → at t=1000 exactly 5 slots free; the 6th is denied with `retryAfterMs: 500`.

EXTENSION (A) · BUILT

Limits per resource and per consumer

Most specific rule wins:

1. **overrides[resource][consumer]**
2. **consumers[consumerId]**
3. **resources[resource]**
4. **default**

```
createRateLimiter({
  policies: new ConfigurablePolicyResolver({
    default:   { limit: 10, windowMs: 1000 },
    resources: { search: { limit: 5, windowMs: 1000 } },
    consumers: { 'partner-acme': { limit: 100, windowMs: 1000 } },
    overrides: { search: { 'partner-acme': { limit: 50, ... } } },
  }),
});
```

- **Validated up front**: `InvalidArgumentError`, e.g. `overrides.search.partner-acme.limit must be a positive integer, got 0` (a zero limit would mean zero-wait denies and a spinning client)
- **Collision-proof keys**: `JSON.stringify([resource, consumerId])`, so `["a:b","c"]` ≠ `["a","b:c"]`
- Own-property lookups: IDs such as `"__proto__"` or `"toString"` are safe
- Missing `resource` → `DEFAULT_RESOURCE` (`"default"`)

INTEGRATION

HTTP headers and client-side waiting

Server: `toRateLimitHeaders(result)`

Header	Value
`RateLimit-Limit`	Quota per window
`RateLimit-Remaining`	Units left
`RateLimit-Reset`	*Relative* seconds until a unit frees up
`Retry-After`	**Only on deny** (RFC 9110)

- IETF RateLimit header fields draft
- Seconds **rounded up**: 1 ms → 1, 1000 ms → 1, 1001 ms → 2, so an obedient client is never early
- The request using the last unit is allowed with `remaining: 0`, no `Retry-After`

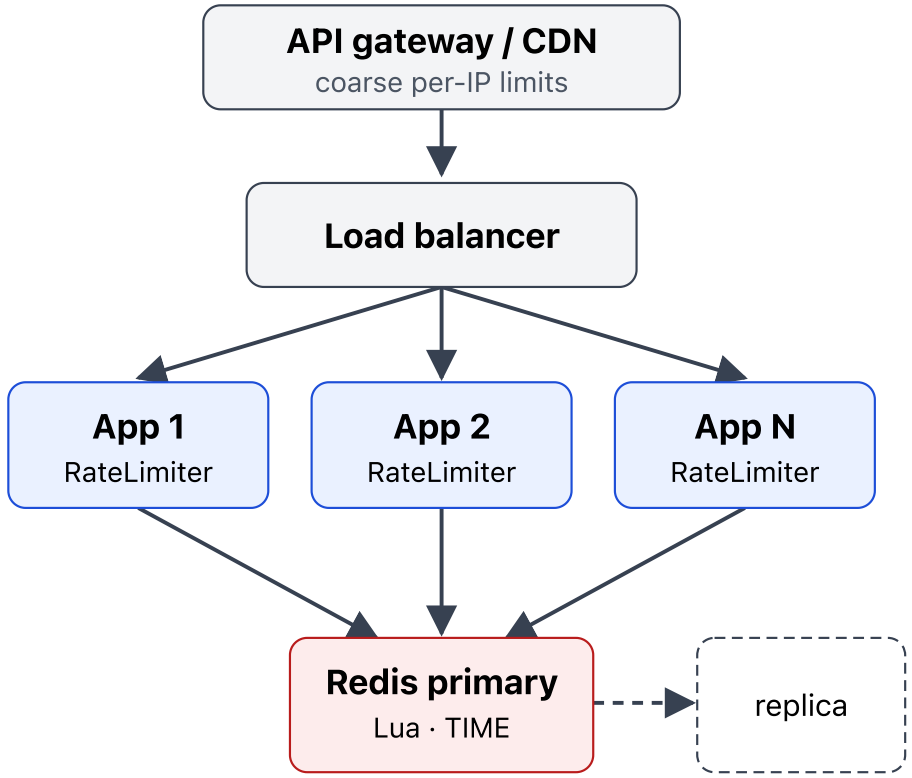
Client: `acquire` / `withRateLimit`

```
await acquire(limiter, { consumerId: 'nightly-job' }, {
  maxWaitMs: 5000, jitterMs: 50,
  signal: AbortSignal.timeout(10_000),
});
const throttled =
  withRateLimit(limiter, consumer, callApi);
```

- Waits the **exact** `retryAfterMs`, not exponential backoff
- `jitterMs` adds 0..jitter *on top*: spread out, never early
- `maxWaitMs` / `maxAttempts`: fail fast with `RateLimitTimeoutError`, without sleeping first
- `signal` aborts at once; invalid options → `InvalidArgumentError`
- Ideas from `urllib3.Retry`, `tenacity`, `p-limit`

EXTENSION (B) · DESIGNED, NOT BUILT

High availability: shared state in Redis



Today: in-memory per process → N × 10/s

Plug-in options

- `RedisStore` behind `RateLimitStore`, atomic via `WATCH` / `MULTI` retries
- Simpler: a Redis `RateLimitStrategy` running one Lua script (`ZREMRANGEBYSCORE`, `ZCARD`, `ZADD`, `PEXPIRE`)

Correctness

- One script = atomic check-and-record: no two instances take the last slot
- Redis `TIME`: app clock skew doesn't matter
- Cluster: hash tags `rl:{consumerId}:…` for multi-key checks

Failure policy

- **Fail-open** keeps serving, drops the limit; **fail-closed** turns a Redis outage into an outage
- Middle ground: short timeout → local `limit / N` + circuit breaker

Operations

- Async replication: failover may briefly over-allow; documented
- Local pre-check denies clearly-over consumers without a round trip
- API is already async: callers don't change

QUALITY ATTRIBUTES

Non-functional requirements

Done: each with a verifiable criterion and a test (`docs/plans/rate-limiter.md`)

NFR	Criterion
Scalability	O(limit) memory and time per active key; idle keys swept
Maintainability	New algorithm / policy source / store = new class; factory is the only place picking concrete classes
Readability	One class per file; API matches design note; README covers every export
Security	Env limits and code policies validated (fail fast); collision-proof keys; no self-lockout; `Retry-After` rounded up; `acquire` bounded
Observability	`limit` , `remaining` , `resetMs` , `retryAfterMs` on every result → standard headers

Deferred **Multi-instance scale**

Extension (b) is design-only: shared `RedisStore` behind the same port

Deferred **Max-keys cap**

A flood of unique IDs grows memory until the sweep; mitigate at the gateway or with Redis

Deferred **Logs / metrics**

No logger port yet; the hook is a Decorator over `RateLimitStrategy`

QUALITY

Testing and type safety

226
tests passing, 19 test files

2,000
seeded random arrivals in the invariant test

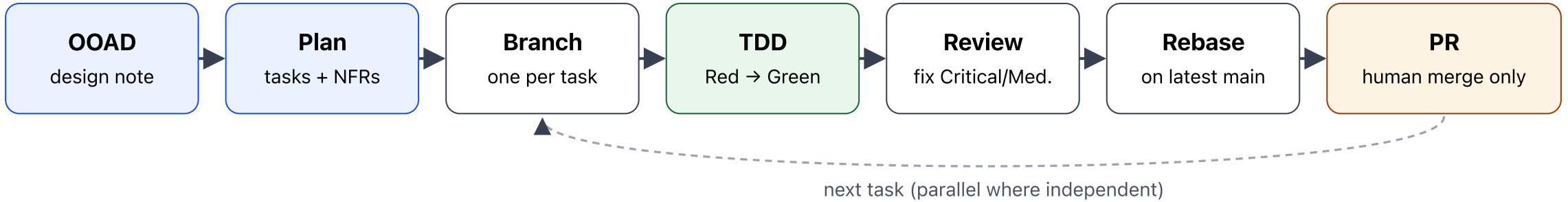
0
type errors under strict `tsc`; no `any`, `as`, `!`, `enum`

14
tasks, one branch + PR each (#8–#25); Red → Green evidence in code PRs

- **TDD**: failing test first, Red output quoted in every code PR
- **Invariant**: no 1-second span ever holds more than 10 allowed requests (`mulberry32(42)`)
- **Deterministic time**: injected `FakeClock`; `vi.useFakeTimers()` only where `setTimeout` runs
- **Tree-shaking probes** (esbuild): barrels stay side-effect free
- **Lint-rule tests**: import bans and ordering really fire
- **Pre-commit hook**: ESLint + Prettier + typecheck; `pnpm check` before every PR

HOW IT WAS BUILT

Engineering process



Design first

Use cases, responsibilities, patterns and a class diagram in `docs/design/` before any code

Small vertical tasks

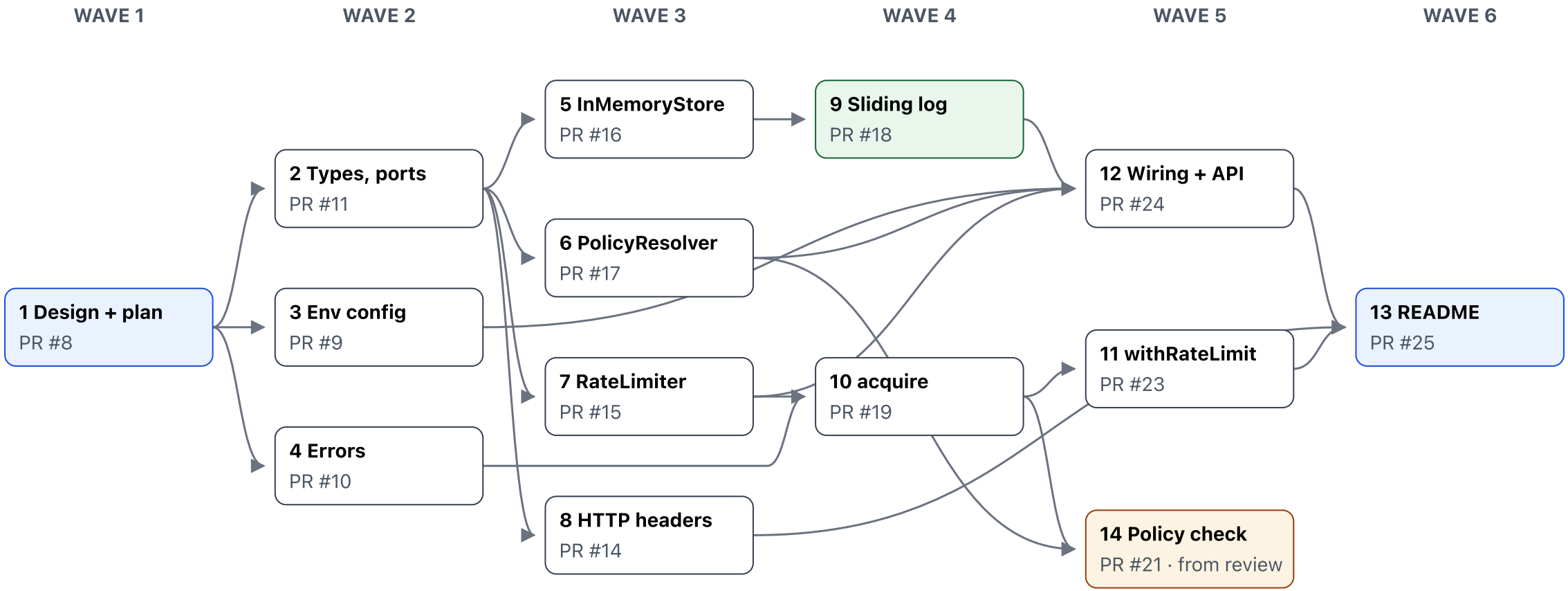
14 tasks, each < ~300 lines with its tests, leaving `main` green on its own

Guardrails

SOLID, GoF only where it removes coupling, Zod-validated env, strict TS, never `--no-verify`

HOW IT WAS BUILT

Delivery: 14 tasks in 6 waves



Same wave = parallel branches. Before: tooling PRs #1–#4, guideline PRs #5, #7, #12. Plan updates: #20, #22.

HONEST GAPS

Known limitations and next steps

Per-process state

- N instances → effective limit **N × limit**
- *Next:* extension (b), Redis behind the existing ports; callers unchanged

Unique IDs grow memory

- Keys live until TTL + next sweep; no max-keys cap
- *Mitigate:* per-IP limits at the gateway, IDs from authenticated identities, or Redis

No logs or metrics

- `LOG_LEVEL` unused; no logger port
- *Next:* a logging Decorator over `RateLimitStrategy` (sketched in the README)

Future improvements (README) · effort S = one PR, M = a few PRs

Scale and HA

- Redis Lua strategy (M)
- Max-keys cap, LRU (S)
- Sliding counter / token bucket (S)

Features

- `strategy` factory option (S)
- Weighted requests (M)
- Policies from DB / plan (M)

Observability

- Logger port + Decorator (S)
- OpenTelemetry metrics (M)

Quality and delivery

- CI on every PR (S)
- Property-based tests (S)
- Benchmarks (S), mutation tests (M)

SUMMARY

Takeaways

- **Correct by construction**: a sliding window log holds "≤ 10 in *any* second", proven by a randomised invariant test
- **Exact answers**: `retryAfterMs` to the millisecond; headers rounded up; denied requests never recorded
- **Extensible**: Strategy, Adapter, Factory and Decorator seams; per-resource / per-consumer limits built
- **Ready to scale out**: async API and store port prepared for Redis (Lua, `TIME`, explicit failure policy)
- **Disciplined delivery**: OOAD → plan → TDD → review → one small PR per task

Questions?